DÉPARTEMENT
de PHYSIQUE

TP N° 3 : Fonctions, graphes et fichiers

Durée esitmée | 2 **séances de** $1h30$

I | But de la manipulation

Le but de ce TP est de manipuler les fonctions, les graphiques et les fichiers avec le langage R?. Les thèmes traités sont les suivants :

1. les fonctions ,
2. les graphiques ,
3. les fichiers.

Des exercices d'application son ensuite donnés à la fin de chaque paragraphe.

(C) Dr. ABBASSI A.
Department of Physics - ISI Mahdia | DM

II | Les fonctions sous R

Le langage R? dispose de fonctions préprogrammées, appelées "primitives". Nous en avons déjà rencontré plusieurs : `c`, `mean`, `sum`, `log`, `sqrt`, etc. L'utilisateur a aussi la possibilité de définir ses propres fonctions lorsqu'elles n'existent pas déjà dans R?. Une fonction est un sous-programme, c'est-à-dire une portion de code qui est exécutée lorsqu'on l'appelle.

De manière générale, la définition d'une nouvelle fonction passe par l'expression suivante :

```
nom_fonction = function(arg1[ = expr1],arg2[ = expr2], ...) {
                    bloc d'instructions }
```

Les accolades marquent à l'interpréteur de commande le début et la fin du code source de la fonction ainsi définie, tandis que les crochets ne font pas partie de l'expression mais indiquent le caractère facultatif des valeurs par défaut des arguments. Il faut ensuite soumettre les fonctions au logiciel, soit en exécutant le bloc de commandes correspondant, soit en utilisant la fonction source.

Lors de l'exécution, R? renvoie par défaut la dernière expression évaluée dans la fonction. On peut également utiliser la fonction return pour spécifier la sortie.

Saisir le script suivant dans l'environnement de RStudio :

```
1
2  power_func = function(x, n) {
3    y <- x^n
4    return(y) #Indique l'objet a retourner
5  } #Indique la fin de la fonction
6
7  power_func(3, 2) #R execute les commandes de la
       fonction carre avec x = 2 et renvoie l'objet
       designe par return
```

Listing 1 – Fonctions dans R

Remarque : Notion de variables locales. A l'intérieur d'une fonction, les variables sont locales : les arguments sont passés à la fonction par valeur et leur portée ainsi que celle de toute assignation classique à l'intérieur d'une fonction est locale. Lors de l'exécution, une copie des

arguments est transmise à la fonction et la fonction ne modifiera pas les valeurs des variables disponibles dans l'environnement de travail. Cependant, on peut utiliser les objets de l'environnement de travail dans les fonctions comme dans l'exemple précédent.

Attention, c'est souvent une source d'erreur de travailler ainsi et il est généralement préférable de déclarer en entrée toutes les variables qui sont utilisées dans la fonction !

Saisir le script suivant dans l'environnement de RStudio :

```
1
2   power_2 = function(x) { #la valeur de n n'est plus
        passee en entree
3    y <- x^n
4    return(y) #Indique l'objet a retourner
5   } #Indique la fin de la fonction
6
7  rm(n) #Efface l'objet n de l'environnement de travail
8  y <- 0
9  power2(5) #Renvoie un message d'erreur car n est inconnu
10 n <- 3
11 power_2(5) #La fonction power est executee avec la
       valeur de n disponible dans l'environnement de
       travail
12 power_2(5)
13 y #La valeur de y n'est pas modifiee (la fonction
       puissance a cree une variable y locale)
```

Listing 2 – Fonctions : variables locales dans R

Un exemple plus avancé. Lorsque la fonction doit retourner plusieurs objets, on utilise généralement une liste comme dans l'exemple ci-dessous. On peut aussi donner des valeurs par défaut à certains paramètres d'entrée.

Saisir le script suivant dans l'environnement de RStudio :

```
1
2   loto = function(a = 5) { #a est le nombre de tirage : 5
        par defaut
3   d <- date() #cree une chaine de caractere avec l'heure
        et la date
4   n <- sample(x = 1:50,size = a) #Vecteur avec a nombres
        au hasard entre 1 et 50
5   print(paste('Tirage du ', d, ':'), quote = FALSE) #
        Affichage de la date
6   print(n) #Affichage des nombres tires
7
8   li <- list(dat = d,alea = n) #Creation de l'objet a
        retourner
9   return(li) #Indique l'objet a retourner
10  } #Indique la fin de la fonction
11
12  res <- loto() #R execute la fonction avec la valeur par
        defaut
13  res$alea
14  loto(8)
```

Listing 3 – Fonctions complexes dans R

Enfin, ajoutons qu'il est fortement recommandé d'ajouter des commentaires au code des fonctions, en les faisant précéder du symbole dièse #. La suite de la ligne est alors ignorée lors de l'interprétation de la fonction et peut donc être complétée par un commentaire libre.

III | Partie pratique

Exercice .1

1. Créer une fonction `myMedian` qui calcule la médiane empirique d'un `n`-écahntillon. On rappelle que si on a `n`-échantillon, $X = (x_1, \cdots, x_n)$ et $\left(x_{(1)}, \cdots, x_{(n)}\right)$ sa statistique d'ordre, alors la médiane se calcul par :

$$MED(X) = \begin{cases} X_{k+1} & \text{si} \quad n = 2k+1 \\ \dfrac{X_k + X_{k+1}}{2} & \text{si} \quad n = 2k, k \in \mathbb{N} \end{cases}$$

2. Comparer avec la fonction pré-programmée de `median` de `R?`.

Exercice .2

1. Créer une fonction `myestvar` qui prend en entrée un vecteur numérique `x` et un argument `biased` (valeur par défaut `FALSE`), et qui donne en sortie lestimateur empirique de la variance calculé sur `x`, sans biais si biased est `FALSE`, biaisé si `biased` est `TRUE` :
   1 En utilisant une structure de contrôle,
   2 En utilisant les opérateurs sur les vecteurs logiques.

IV | Les graphiques sous R

La méthode basique pour créer un graphique est d'utiliser la commande `plot(x, y)` avec `x` un vecteur qui contient l'abcisse des points à représenter et `y` un autre vecteur qui contient l'donnée des points à représenter. La commande `plot` permet aussi de fixer les limites des axes, de donner des noms aux axes et à la figure, etc. Une aide détaillée sur les deux options peut être obtenue en tapant la commande plot.default ?. On peut ensuite rajouter des points ou des lignes avec les commandes points ou lines.

Saisir le script suivant dans l'environnement de RStudio :

```
1
2   vect_x <- 1:10
3   vect_y <- vect_x * vect_x
4   plot(vect_x, vect_y)
5   plot(vect_x, vect_y, type = 'l') #La courbe obtenue est
        constituee de segments.
6
7   vect_x <- seq(1, 10, by = .01) #Il faut discretiser
        plus finement pour que le graphique obtenu
        ressemble a une parabole
8   vect_y <- vect_x * vect_x
9   plot(vect_x, vect_y, type = 'l')
10  plot(vect_x, vect_y, type = 'l', col = 'red') #Couleurs
        classiques : red, blue, cyan, yellow, green,...
11
12  colors() #Liste des couleurs disponibles
13  plot(vect_x, vect_y, type = 'l', col = 'red', xlab ='x'
        , ylab = 'cos(x)', main = 'Titre')
14  #N'oubliez pas de donner des noms a vos axes pour les
        rapports
```

Listing 4 – plot : Les graphiques dans R

Utilisation des fonctions points et lines pour ajouter des éléments sur un graphique existant : Saisir le script suivant dans l'environnement de RStudio :

```
1
2   vect_y2 <- exp(vect_x)
3   points(vect_x, vect_y2, col = 'blue') #Points permet de
        rajouter des points sur le graphique
4   points(vect_x, vect_y2, col = 'blue', pch ='o')
5
6   lines(vect_x, vect_y2, col = 'gray') #Lines permet de
        rajouter des lignes sur le graphique
7   plot(vect_x, vect_y2, type = 'l', col = 'green') #
        Verifier qu'on peut passer d'une fenetre graphique
        a l'autre
8
9   colors() #Liste des couleurs disponibles
10  par(mfrow = c(1,2)) #Divise la fenetre graphique en 1*2
        sous-fenetres
11  plot(vect_x, sin(vect_x), type='l')
12  plot(vect_x, cos(vect_x), type = 'l')
```

Listing 5 – plot : Les graphiques dans R

Gestion des différentes fenêtres graphiques. Avec `RStudio?`, une nouvelle fenêtre graphique est créée à chaque exécution de la commande `plot` et les anciennes fenêtres sont conservées. Les commandes précédentes ont donc créé plusieurs fenêtres graphiques différentes.

Vous pouvez passer d'un graphique à l'autre en utilisant les flèches situées au dessus du graphique. Il faut aussi penser à supprimer les graphiques inutiles pour éviter de surcharger la session `R?`. Pour cela, on peut les boutons situés au dessus de la fenêtre graphique. Par exemple,

(C) Dr. ABBASSI A. - Univ. of Physics - DÉPARTEMENT de PHYSIQUE

en cliquant sur le balai on efface tous les graphiques existants. On peut aussi utiliser la commande en ligne `dev.off()`.

Utilisation des fonctions existantes dans `R?` pour faire des graphiques spécifiques. Nous verrons par la suite que `R?` contient des fonctions (`hist`, `pie`, `boxplot`, `qqplot`,`etc`) pour faire certains graphiques spécifiques (respectivement histogrammes, camemberts, boîtes à moustache, 'Quantile-Quantile' plot,...). Nous verrons également qu'il existe des packages dédiés à la réalisation de graphiques : par exemple `ggplot2` pour les statistiques descriptives ou `leaflet` pour les cartes.

V | Partie pratique

Exercice .1

1. Tracer la fonction `sin` sur l'intervalle `[0, 10]` ;
2. Reprendre le graphique précédent en imposant les limites `[-1.5, 1.5]` pour l'axe des ordonnées (option `ylim`). On donnera le titre `Graphique de la fonction sinus` au graphique, le nom `Angle` à l'axe des abscisses et le nom `Sinus` à l'axe des ordonnées ;
3. Rajouter la droite d'équation $y = \frac{x}{2\pi}$ sur le graphique précédent à l'aide la fonction lines.
4. Rajouter les points de coordonnées `(0, 1.3)` et `(2, -1)` sur le graphique précédent à l'aide de la fonction `points`. On représentera ces points à l'aide du symbole `*` colorié en bleu. On tracera cette droite en rouge.

Exercice .2

1. Séparer la fenêtre graphique en `2 x 2` parties puis tracer :
   1 la fonction cos en haut à gauche,
   2 la fonction sin en haut à droite,
   3 la fonction exp en bas à gauche,
   4 la fonction ln en bas à droite.

   On donnera des titres aux différents graphiques et des noms aux axes.

VI | Fonctions utiles

Cette section regroupe quelques fonctions utiles en **statistique** et en **analyse numérique**. Nous allons travailler avec la base women de `R?`.

```
1
2   # Affichage de la base :
3   women ; head(women) # les premiere lignes de la base
4   mean(women$height) #Calcul de la moyenne empirique de
        la variable height
5   var(women$height)  #Calcul de la variance empirique (
        estimateur non biaise)
6
7   sd(women$height) #Calcul de l'ecart-type de height
8   median(women$height) #Calcul de la mediane empirique
9   quantile(women$height) #Calcul des quantiles empiriques
10
11  summary(women$height) #Resume de la variable height
12  summary(women) #Resume de la base women
13  hist(women$height, nclass = 5) #Histogramme de weight
14  boxplot(women$weight) #Diagramme en boite de weight (
        boit a moustache)
15  cor(women$height, women$weight) #Calcul du coefficient
        de correlation lineaire empirique entre weight et
        height
16
17  vector_1 <- rnorm(100) #Simulation de valeurs selon un
        loi de Gauss (cf cours de proba !)
18  hist(vector_1)
19  vector_2 <- factor(sample(letters[1:4], 100, rep = T))
20  table(vector_2) #Tableau d'effectifs
21  barplot(table(vector_2)) #Diagramme en barre de vector_
        2
22
```

(C) Dr. ABBASSI A. Department of Physics

```r
pie(table(vector_2)) #Diagramme circulaire de vector_2
boxplot(vector_1 ~ vector_2) #Diagramme en boite de
    vector_1 pour chaque modalite de vector_2
```

Listing 6 – Statistque : Fonctions utiles de R

```r

my_function = function(x){ #Definition de la fonction a
     etudier
               return(sin(x)/x)
}
plot(my_function, -20, 20)
integrate(my_function,0,5) #Calcul approche de l'
    integrale
optim(.1, my_function) #Recherche d'un minimum local
    autour de x = .1
res <- optim(.1,  my_function) #res est une liste
points(res$par, res$value) #Representation graphique
res <- optim(10,  my_function) #Recherche d'un minimum
    local autour de x = 10
points(res$par, res$value) #Representation graphique
uniroot(my_function, lower = .1, upper = 6) #Recherche
    d'une solution de l'equation f (x) = 0 sur l'
    intervalle x dans [.1, 6]
```

Listing 7 – Analyse numérique : Fonctions utiles de R

(C) Isi Mahdia - Dpt. Physique - ISI Mahdia | UMA

VII | Partie pratique

Exercice .1

On considère dans cet exercice le jeu de données `iris` disponible dans `R?`.

1. Réaliser un histogramme de la taille des `sépales`.
2. Proposer un graphique adapté permettant de représenter la distribution de la taille des `sépales` pour les différentes espèces d'`iris`.

Exercice .2

On reprend dans cet exercice les données de l'exercice `1p4-5 : TP 2`, à l'aide de `diagrammes circulaires` :

1. Représenter les distributions du nombre d'étudiants par `discipline`.
2. Représenter les distributions du nombre d'étudiants par `académie`.

VIII | Les packages R

Un `package`[1] est un ensemble de programmes qui complète les fonctionnalités de `R?`[2]. L'installation d'un package se fait en deux étapes :

1 Une fois le package cible fixé, on peut utiliser la commande `install.packages` ou l'outil `Install Packages` du menu `Tools`[3] ;

2 Ensuite charger le package dans la session `R?`. Pour cela, on peut utiliser la commande `library` ou le gestionnaire de package de `RStudio?` (onglet dans la fenêtre en bas à droite par défaut)[4].

IX | Partie pratique

Exercice .1

Après avoir installé le `package MASS`.

1. ou bibliothèque ou extension)
2. Il existe un très grand nombre de packages qui sont diffusées par un réseau appelé **CRAN** (**C**omprehensive **R** **A**rchive **N**etwork) : https ://cran.r-project.org/web/packages/
3. On n'a besoin d'effectuer cette opération qu'une seule fois
4. effectuer cette opération à chaque début de session ou de script.

1. Simuler un échantillon de taille `n = 100` d'une `loi Exponentielle` de paramètres λ `= 2` avec la fonction `rexp`. Faire un histogramme de l'échantillon simulé.
2. On suppose maintenant que λ est inconnu. Utiliser la fonction `fitdistr` pour estimer λ et superposer la densité correspondante sur l'histogramme de la question précédente.

Exercice .2

Après avoir installé le package `leaflet`, représenter sur une même carte votre lieu de naissance et l'`EURIA`.

X | Les entrées et sorties sous R | les fichiers

Dans de nombreux cas, les données que l'on souhaite analyser proviennent de sources externes sous forme de fichiers. Aussi, les objets créés doivent pouvoir être sauvegardés dans des fichiers afin qu'ils soient complètement transportables. C'est ainsi que divers outils d'accès aux fichiers ont été développés sous `R?`. On distingue les accès aux fichiers propriétaires de `R?`, aux fichiers `ASCII`, aux fichiers provenant d'autres **logiciels d'analyse statistique**, aux **bases de données relationnelles** et aux fichiers **binaires**.

On distingue :

1 **Fichiers de données délimitées**[5] : ceux sont des fichiers texte structurés, souvnet tabulaires, stockés localement. On trouve les formats suivants : `.csv`, `.tsv`, `.txt`, `.fwf` (largeur fixe). Les fonctions utilisées pour lire ces fichiers sont : `read.csv()`, `read.table()`, `read.delim()`, `readr::read_csv()`, `readr::read_tsv()` ;

2 **Ficiers Excel** : ceux sont des feuilles de calcul Excel, souvent utilisées dans les entreprises. On trouve les formats suivants : `.xls` et `.xlsx`. Les fonctions utilisées sont : `readxl::read_excel()`, `openxlsx::read.xlsx()` ;

3 **Bases de données :** systèmes de gestion de données relationnelles ou `NoSQL`. On a les types : `MySQL`, `PostgreSQL`, `SQLite`, `Oracle`, `MongoDB`. Les packages `R?` utilisés sont : `DBI`, `RMySQL`, `RPostgreSQL`, `RODBC`, `RMongo` ;

4 **Données à partir du Web** : ceux sont des données accessibles via internet : `API`, pages web, fichiers distants. Les sources sont `JSON`, `HTML`, `XML`, `CSV` en ligne. Les packages `R?` sont : `httr`, `jsonlite`, `rvest`, `curl` ;

5 **Logiciels de statistiques** : ceux sont des fichiers de données générées par des logiciels des statistiques. Les formats disponibles sont : `.sav` (SPSS), `.dta` (Stata), `.sas7bat` (SAS). Les packages `R?` utilisés sont : `haven`, `foregin`.

Dans cette partie on se limitera aux fichiers : `.csv`, `.txt` et `.xls | .xlsx`.

```
1
2   # Fichier .csv
3
4   Exemple 1 :
5
6   # Telecharger un fichier .csv depuis GitHub
7   url_csv <- "https://raw.githubusercontent.com/
        datasciencedojo/datasets/master/titanic.csv"
8   data_csv <- read.csv(url(url_csv))
9
10  # Apercu des donnees
11  head(data_csv)
12
13  Exemple 2 :
14
15  # Import swimming_pools.csv: pools
16  pools <- read.csv("http://s3.amazonaws.com/assets.
        datacamp.com/production/course_1477/datasets/
        swimming_pools.csv",
17  stringsAsFactors = TRUE)    #
18
```

5. Flat files ou fichiers plats

(C) Dr. ABBASSI A. Department of Physics | UM ISI Mahdia

```r
# Print the structure of pools
str(pools)

# Exemple 1 :
# Fichier .txt

# Telecharger un fichier .txt depuis GitHub
url_txt <- "https://raw.githubusercontent.com/
    justmarkham/DAT8/master/data/u.user"
data_txt <- read.delim(url(url_txt), sep = "|", header
    = TRUE)

# Apercu des donnees
head(data_txt)

# Example 2 :
# Import hotdogs.txt: hotdogs
hotdogs <- read.delim("http://s3.amazonaws.com/assets.
    datacamp.com/production/course_1477/datasets/
    hotdogs.txt", header
= FALSE)

# Summarize hotdogs
summary(hotdogs)

# Path to the hotdogs.txt file: path

path <- file.path("http://s3.amazonaws.com/assets.
    datacamp.com/production/course_1477/datasets", "
    hotdogs.txt")

# Import the hotdogs.txt file: hotdogs
hotdogs <- read.table(path,
+                       sep = "\t",
+                       col.names = c("type", "calories
    ", "sodium"))

# Call head() on hotdogs
head(hotdogs)
```

Listing 8 – Fichiers sous R

Remarque 1 N'hésiter pas à visiter le **site officiel de R Project**, et découvrir la documentation disponible : https ://www.r-project.org/.

(C) Dr. ABBASSI - Department of Physics - ISI Mahdia